

Claude Certified Architect: The Complete Guide to Passing the CCA-F Foundations Exam

Everything You Need to Know to Ace the CCA Foundations Exam on Your First Try: Article 1 of 8



RICK HIGHTOWER

JUN 02, 2026



5



2

Share

Mastering the Claude Certified Architect (CCA) Foundations Exam

This certification tests the ability to architect production-ready systems using Claude, focusing on agentic architecture, tool integration, and programmatic reliability rather than just prompt engineering.

COMPETENCY DOMAINS & LOGISTICS

Agentic Architecture (27%)

Focus on multi-agent orchestration and explicit context passing.

MCP: Tools vs. Resources (18%)

Tests distinguishing executable actions (Tools) from read-only context (Resources).



Executable Actions (Tools)



Read-only Context (Resources)

Programmatic Enforcement Over Prompts (20%)

20% of the exam rewards using `tool_choice` and validation loops instead of prompt instructions.

EXAM LOGISTICS



Format:
60 Multiple-Choice Questions



Time Limit:
120 Minutes



Passing Score:
720 / 1,000

THE 4-WEEK STUDY STRATEGY

Week 4: Simulation & Anti-Patterns

Take the practice exam and memorize "what NOT to do" patterns.

Week 3: MCP & Claude Code

Create an MCP server and configure CLAUDE.md files for project-level standards.

Weeks 1-2: Build the Core

Focus on building validation-retry loops and mastering the `tool_use` stop reason.

On March 12, 2026, Anthropic launched the Claude Certified Architect (CCA) Foundations exam. The AI industry finally has a professional certification that tests whether you can actually build production systems with Claude. Not whether you can write clever prompts. Not whether you watched a tutorial. Whether you can architect real software that runs in production.

If you are preparing for the Claude Certified Architect exam and want to pass on your first attempt, this is the guide. This is article 1 in an eight-part series. I am going to walk you through everything you need to pass on your first attempt: the exam format, all five competency domains, the six production scenarios, the mental models that separate passing candidates from failing ones, and a week-by-week study plan built around where people actually lose points. The remaining seven articles deep-dive the hardest scenarios so you can dissect each one before exam day.

Hightower's AI Harness Engineering is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Subscribe

Let me be direct about what you are signing up for. Anthropic calls this:

“301-level exam designed for seasoned professionals with at least 6 months of direct hands-on experience building with Claude.”

The questions are scenario-based, the distractors are plausible, and the ***exam tests what NOT to do as much as what to do***. If you have been building with the Claude API and Claude Code for a while, you already know most of this material. The exam tests whether you know it precisely enough to pick the right answer under time pressure.

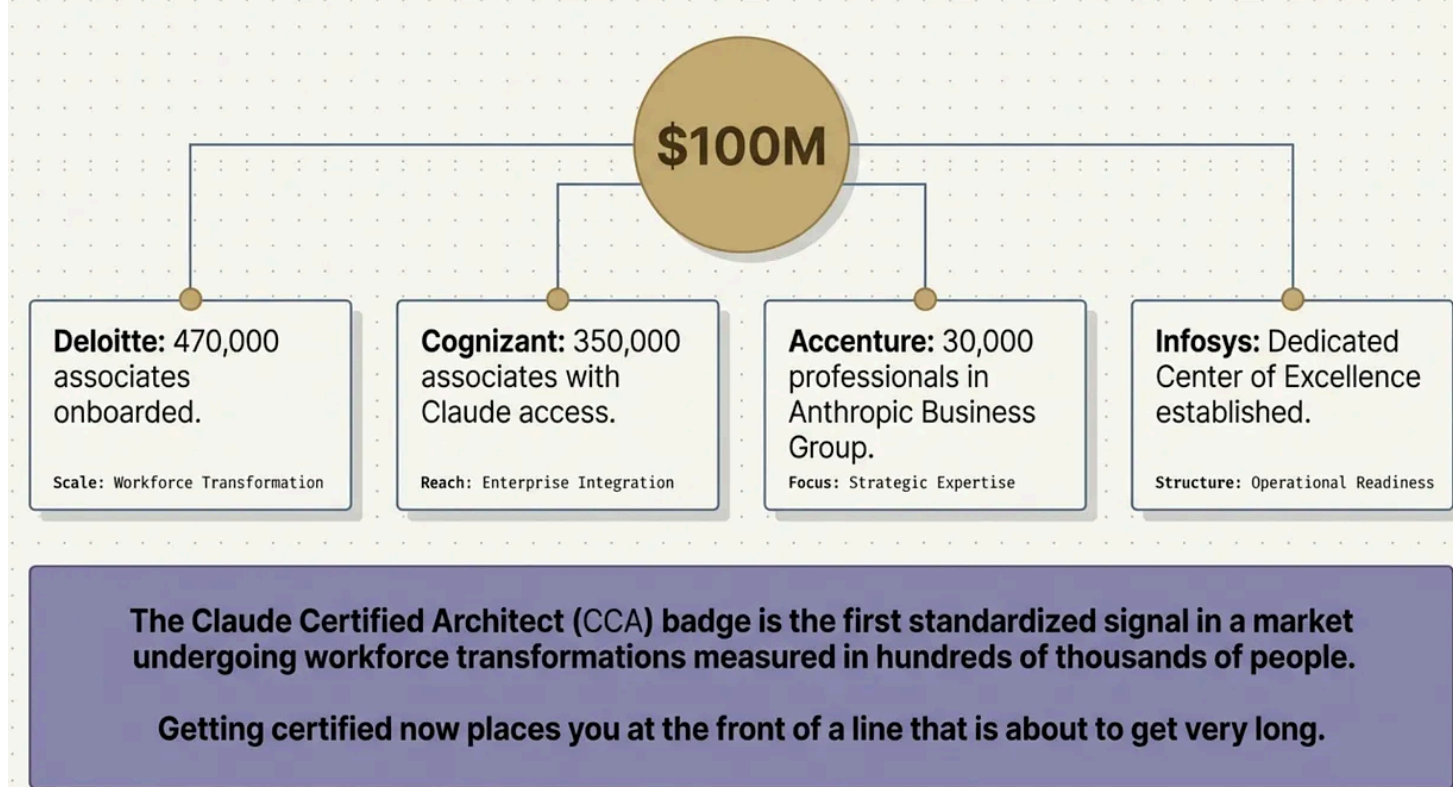
The exam tests what NOT to do as much as what to do.

The exam tests what NOT to do as much as what to do.

Why the CCA Matters Right Now

Anthropic backed this certification with a [\\$100 million Claude Partner Network investment](#). Let that number land for a moment, because it tells you exactly what this credential is worth. A \$100M commitment does not come without enterprise demand at scale.

Look at the partner commitments Anthropic announced alongside that investment. Accenture trained 30,000 professionals and launched a dedicated Anthropic Business Group. Cognizant gave 350,000 associates Claude access. Deloitte put 470,000 associates on the platform. Infosys stood up a Center of Excellence. These are not pilot programs. These are workforce transformations measured in hundreds of thousands of people.



Accenture, Cognizant, Deloitte, Infosys investing heavily

Here is the implication for you: every one of those organizations needs people who can architect Claude-based systems at a professional level. The credential gives hiring managers a standardized signal in a market that currently has no standardized signal. You showing up with a CCA badge while the market is still forming is a different proposition than showing up two years from now when everyone has one.

The CCA is the first certification Anthropic has ever offered. Additional certifications for sellers, architects, and developers are planned for later in 2026. Getting certified now, while the community is small and the credential is fresh, puts you at the front of a line that is about to get very long.

There is also a more personal reason to pursue it: the preparation process alone will close gaps in your knowledge that you did not know existed. Even if you have been building with Claude for months, the exam will surface edge cases in your understanding of tool design, context management, and agentic architecture that hands-on work alone does not reveal.

You showing up with a CCA badge while the market is still forming is a different proposition than showing up two years from now when everyone has one.

Exam Format: What You Are Walking Into

Here is everything you need to know about logistics before you start studying.

Detail	Value
Full Name	Claude Certified Architect: Foundations
Questions	60 multiple-choice
Time Limit	120 minutes
Scoring	100 to 1,000 scale
Passing Score	720 (roughly 72nd percentile)
Cost	\$99 per attempt (free for first 5,000 partner employees)
Eligibility	Claude Partner Network members (organizations join for free)
Proctoring	Online via ProctorFree
External Resources	None. No Claude, no docs, no browser tabs.
Results	2 business days, detailed breakdown + shareable badge
Registration	anthropic.skilljar.com — https://anthropic.skilljar.com/claude-certified-architect-foundations-access-request

A few things matter here more than the table suggests.

Two minutes per question is tight when the question is a paragraph. These are not vocabulary questions. Each question describes a production scenario, sometimes in 150–200 words, and then asks you to choose the best architectural decision. You need to read fast and pattern-match faster. Candidates who score in the 900s report that their speed came from recognizing trap patterns instantly, not from reading every word carefully.

You cannot pause the exam once it starts. Handle your logistics beforehand. Quiet room, water, bathroom break, a machine that will not run out of battery. It is proctored so the assumption is that you will use their equipment (the company that is doing the proctoring).

Four of six production scenarios appear on your exam. The four scenarios are drawn randomly, so you cannot skip any scenario during preparation. Every scenario is fair game. Study all six.

You are completely on your own. No Claude, no documentation, no external tools. The exam tests internalized knowledge. This is why building real systems during your study period matters so much: the knowledge has to live in your head, not in a browser tab.

Time management strategy from early test-takers: do a quick first pass and flag any question that makes you pause. Answer the ones you know cold. Then return to the flagged questions with your remaining time. Do not burn five minutes on a single question when 58 others need attention.

The Crucible: Zero Resources, 120 Minutes

120 Minutes 60 Scenario-Based Questions	720 Passing Score Top ~28% percentile	\$99 Per Attempt ProctorFree online
--	--	--

 **ZERO EXTERNAL RESOURCES.** No Claude. No documentation. No browser tabs. The knowledge must live entirely in your head.

Tactical Brief

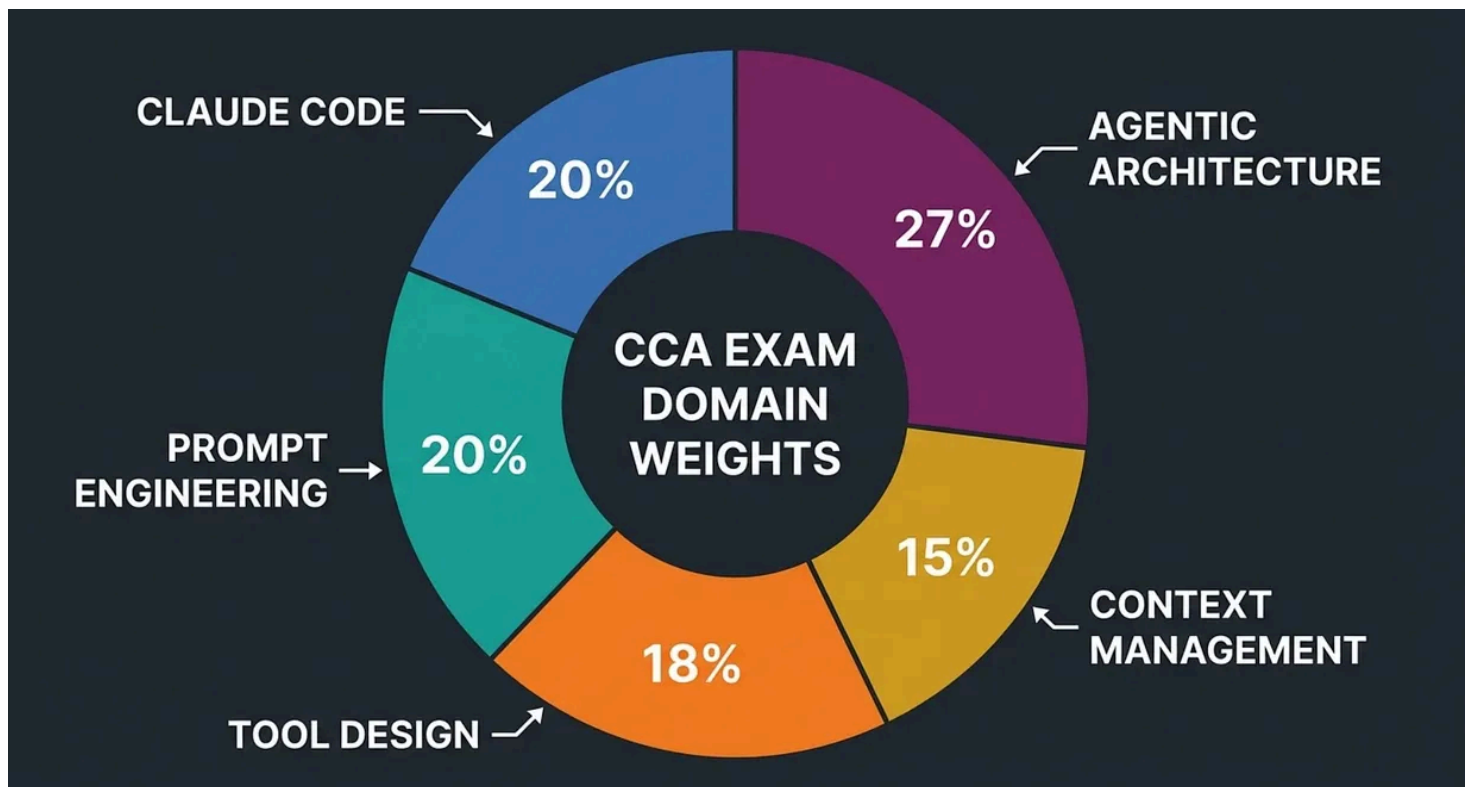
Two minutes per question is severely tight when reading 150-200 word production scenarios. Speed comes from recognizing trap patterns instantly, not from reading every single word carefully. Flag paralyzing questions and move on.



120 minutes, 60 questions, must get 720 out of 1,000 and the test cost \$99

The 5 Competency Domains

Every question maps to one of five domains. The weights tell you exactly where to invest your study time. Do not treat all five domains equally. Domain 1 alone is worth more than Domain 5. Start there.



CCA exam domain weight distribution chart showing five segments: Agentic Architecture 27%, Claude Code 20%, Prompt Engineering 20%, Tool Design 18%, Context Management 15%

Domain 1: Agentic Architecture and Orchestration (27%)

This is the heavyweight. More than a quarter of the exam tests your ability to design multi-agent systems. If you underinvest here, you cannot make it up elsewhere.

What you need to know:

- **Multi-agent patterns:** the coordinator-subagent pattern (coordinator delegates tasks to specialized subagents and synthesizes their results) and hub-and-spoke (parallel independent tasks with no dependencies between them). Know when each is appropriate and when each fails.
- **The critical rule that trips most candidates:** subagents do NOT inherit context automatically. When a coordinator spawns a subagent, the subagent starts with a blank context. You must explicitly pass everything it needs. This sounds obvious

written down, but the exam tests it in scenarios where candidates assume context flows naturally between agents. It does not.

- **Session state management:** how to maintain conversation context across turns in an agentic loop. Where does state live? How do you prevent context from growing unbounded?
- **Task decomposition:** breaking complex requests into discrete subtasks that individual agents can handle. The exam tests your ability to identify the right granularity.
- **Agentic loop design:** the cycle of receiving input, reasoning, calling tools, evaluating results, and deciding whether to continue or return. Know every step.
- **Escalation logic:** when and how an agent should hand off to a human or a more capable system. The exam rewards deterministic escalation rules over model-driven escalation.

💡 **Exam Tip:** The most common Domain 1 trap is the “super agent” anti-pattern: one agent with 15+ tools instead of 3–4 specialized agents with 4–5 tools each. When you see an answer choice that consolidates everything into a single powerful agent, that answer is almost always wrong.

💡 **Study strategy:** Build a small multi-agent system using the Claude Agent SDK. Have a coordinator spawn two subagents, pass them different contexts explicitly, and synthesize their results. When you struggle to pass context correctly in real code, you will never forget the rule on the exam.

Domain 2: Claude Code Configuration and Workflows (20%)

One-fifth of the exam covers Claude Code, Anthropic’s agentic coding tool.

What you need to know:

- [CLAUDE.md](#) hierarchy: project-level (`.claude/CLAUDE.md`, shared via version control) versus user-level (`~/ .claude/CLAUDE.md`, personal, not shared). Think of the project-level [CLAUDE.md](#) as the “tech lead” file that sets project standards for the whole team. User-level is your personal customizations. The exam tests whether you know which file belongs in version control.

- **Custom slash commands and skills:** how to create reusable workflows as markdown-based skills. Know the structure, not just that they exist.
- **Plan mode versus direct execution:** plan mode is for complex, multi-step tasks where you want Claude to lay out the approach before touching anything. Direct execution is for well-defined, lower-risk tasks. The exam tests your judgment about which mode is appropriate for a given scenario.
- **CI/CD integration:** the `-p` (or `--print`) flag for non-interactive pipelines. Forgetting the `-p` flag in CI/CD is a heavily tested anti-pattern. The flag switches Claude Code from interactive to headless mode.
- **The `--bare` flag:** for scripted CI/CD calls, `--bare` skips auto-discovery and ensures reproducible behavior. Anthropic's docs say this will become the default for `-p` in a future release. Know it.
- **`--output-format json`:** for structured JSON output from Claude Code, supporting `text`, `json`, and `stream-json` modes. When your CI/CD pipeline needs to parse Claude Code output, this flag is the correct tool.

💡 **Exam Tip:** The CI/CD questions almost always have a trap answer that involves running Claude Code interactively or without the `-p` flag. The moment you see a CI/CD context, mentally check whether the `-p` flag is present. If it is missing from the proposed solution, that solution is wrong.

💡 **Study strategy:** Set up a [CLAUDE.md](#) in a real project. Run Claude Code in a CI/CD pipeline with the `-p` flag. Create a custom slash command. The exam questions assume you have done these things, not just read about them.

Domain 3: Prompt Engineering and Structured Output (20%)

This domain tests your ability to get reliable, structured output from Claude in production systems. The key word is “reliable.” Claude producing JSON most of the time is not good enough for production. The exam tests whether you know the difference between approaches that produce reliable output and approaches that produce unreliable output.

What you need to know:

- **JSON schema enforcement with `tool_choice`:** using `tool_choice` with input schemas to force structured output. This is not asking Claude to produce JSON in the system prompt. This is using the API's tool mechanism to constrain the output format programmatically. Learn the difference.
- **The structured outputs API:** `client.messages.parse()` with Pydantic models (beta feature, requires `anthropic-beta: structured-outputs-2025-11-13` header). Know that this exists and when to use it.
- **Few-shot prompting:** providing example inputs and outputs to guide Claude's response format. Useful, but not a replacement for programmatic enforcement.
- **Validation-retry loops:** check the output against your schema. If it fails, send the error back to Claude with instructions to fix it. The `tool_use` stop reason is your signal to inspect and validate. Build this loop; do not just read about it.
- **The core anti-pattern:** relying on prompts alone for JSON compliance. "Please respond only in valid JSON" in your system prompt will work most of the time. The exam will give you scenarios where "most of the time" causes failures and ask you to fix them. The correct fix is programmatic enforcement, not a better prompt.

💡 **Exam Tip:** Any answer choice that says "add instructions to the system prompt" or "include more detailed formatting guidance in the prompt" as the solution to JSON compliance is almost certainly wrong. The exam consistently rewards `tool_choice`, structured outputs API, and validation-retry loops over prompt-only approaches.

💡 **Study strategy:** Build a validation-retry loop. Send Claude a schema, get output, validate it, and retry on failure. Then build the same thing using `tool_choice`. Notice how much more reliable the `tool_choice` approach is. That experience makes exam questions in this domain feel obvious.

Domain 4: Tool Design and MCP Integration (18%)

The Model Context Protocol (MCP) is Anthropic's open standard for connecting AI systems to external tools and data. Launched in November 2024, MCP has been adopted across the industry; both OpenAI and Google DeepMind have integrated it. This domain is where community feedback says most candidates lose the most unexpected points.

What you need to know:

- **MCP primitives and when to use each:**
- **Tools:** executable functions the model can invoke. Use for actions: querying a database, calling an API, writing a file.
- **Resources:** data for ingestion into prompts or RAG pipelines. Use for read-only context: documentation, schemas, knowledge bases.
- **Prompts:** predefined templates or workflows. Use for reusable instruction patterns.

The exam tests the boundaries between these three. Getting this wrong is the single most common source of lost points on Domain 4.

- **Configuration files:** `.mcp.json` is project-level and shared via version control; `~/.claude.json` is user-level and personal. This mirrors the [CLAUDE.md](#) hierarchy exactly. One pattern to remember both.
- **Tool descriptions as routing:** the description you write for a tool is the primary mechanism Claude uses to decide which tool to call. Agent names do not matter for routing. Tool names matter less than descriptions. Write tool descriptions as if you are writing documentation for a developer who has never seen your codebase.
- **The 4–5 tool rule:** Anthropic’s official guidance is 4–5 focused tools per agent. When an agent has 18 tools, selection reliability degrades measurably. Distribute excess tools to specialized subagents.

💡 **Exam Tip:** The tool-versus-resource boundary is the hardest judgment call on this domain. When an exam question asks whether something should be a tool or a resource, ask yourself: does Claude need to invoke it to take an action, or does it just need the data as context? Actions are tools. Context is resources. If it is read-only data that helps Claude understand something, it is a resource. If Claude needs to execute it to make something happen, it is a tool.

💡 **Study strategy:** Build an MCP server in Python using the official SDK. Expose 3–4 tools and a resource. Connect it to Claude Code. The MCP Mastery course on Anthropic Academy covers this end-to-end. You need to have done this to answer tool boundary questions with confidence.

Domain 5: Context Management and Reliability (15%)

The smallest domain by weight, but it cross-cuts every scenario. Understanding context management poorly means you get questions wrong in Domains 1, 2, 3, and 4 as well. Study this domain more than its 15% weight suggests.

What you need to know:

- **The “lost in the middle” effect:** Claude and all transformer-based LLMs attend more to information at the beginning and end of the context window. Content buried in the middle receives less attention. Place critical information at the start or end of your context. This is not theoretical; it affects production agent performance in measurable ways.
- **Context degradation in long sessions:** as conversations grow, earlier context becomes less reliable. The exam tests whether you know how to mitigate this: structured summaries, periodic restatement of critical facts, keeping the most important information anchored at the start.
- **Escalation design:** when an agent should stop trying and hand off. The exam tests your judgment about escalation thresholds. The pattern is always: deterministic rules trigger escalation, not model self-assessment.
- **Token economics:** know the differences cold.

Feature	Cost Savings	Latency	Use Case
Prompt Caching	Up to 90%	Real-time	Repeated system prompts, policy documents, few-shot examples
Message Batches API	50%	Up to 24 hours (most within 1 hour)	Nightly audits, bulk processing, offline workloads
Real-Time API	Standard pricing	Real-time	User-facing, blocking workflows

Token Economics

Prompt Caching offers up to 90% cost savings with real-time latency, making it ideal for repeated system prompts, policy documents, and few-shot examples. This approach

allows you to cache frequently used context and dramatically reduce costs while maintaining immediate response times.

Message Batches API provides 50% cost savings but with latency of up to 24 hours, though most requests complete within 1 hour. This API is designed for nightly audits, bulk processing, and offline workloads where immediate responses are not required.

Real-Time API operates at standard pricing with real-time latency, serving user-facing and blocking workflows where users are actively waiting for responses. This is the default choice when cost optimization cannot come at the expense of user experience.

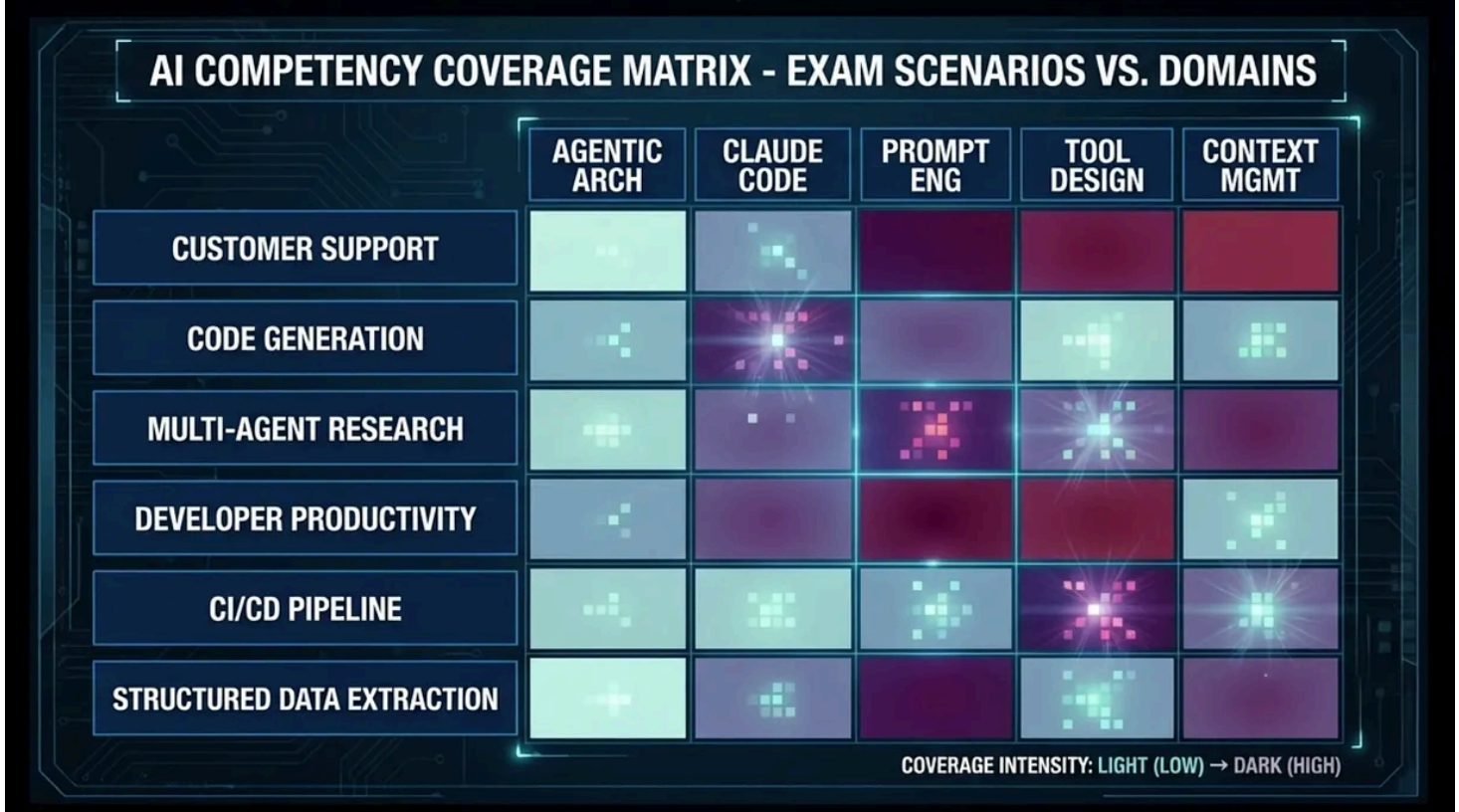
Using the Batch API for a live user-facing workflow is an anti-pattern. Using the real-time API for overnight bulk processing wastes money. The exam tests your ability to match API to use case.

Exam Tip: When you see a question about cost optimization in a system where users are actively waiting for responses, the answer is never the Batch API. The answer is Prompt Caching for repeated context. When you see a system doing overnight processing or batch audits, the Batch API is almost always the right cost optimization.

Study strategy: Experiment with context window limits. Send Claude a long document with critical information in the beginning, middle, and end. Test whether the responses treat all three sections equally. The difference in behavior will stick with you during the exam.

The 6 Production Scenarios

Every exam draws 4 of these 6 scenarios at random. You cannot predict which four, so you must prepare for all six. Each scenario presents a realistic production situation and tests your architectural judgment across multiple domains.



CCA scenario-domain coverage matrix mapping six exam scenarios to five competency domains with intensity shading

#	Scenario	Primary Domain	Key Trap	Difficulty
1	Customer Support Resolution Agent	Agentic Architecture	Using self-reported confidence for routing decisions	Hard
2	Code Generation with Claude Code	Claude Code Workflows	Assuming a bigger context window solves everything	Medium
3	Multi-Agent Research System	Agentic Architecture	Building one super-agent with all 18 tools	Hard
4	Developer Productivity Tools	Claude Code Workflows	Ignoring CLAUDE.md configuration hierarchy	Medium
5	Claude Code for CI/CD	Claude Code + Prompt Eng	Forgetting the -p flag for non-interactive mode	Medium
6	Structured Data Extraction	Prompt Engineering	Relying on prompt-only enforcement for JSON output	Medium

Scenarios covered

Here is a brief preview of each. The remaining articles in this series go deep on the hardest ones.

Scenario 1: Customer Support Resolution Agent. You are building a support agent using the Claude Agent SDK that handles high-ambiguity requests: returns, billing disputes, account issues. The scenario tests escalation logic, compliance constraints, and the financial consequences of wrong routing. *The biggest trap: using Claude’s self-reported confidence score to decide whether to escalate. LLM-generated confidence scores are not calibrated for production routing decisions.* Use programmatic rules instead: ticket category, dollar amount, account tier, issue type. These rules live in code, not in prompts.

Scenario 2: Code Generation with Claude Code. You are configuring Claude Code for large codebase navigation and code generation tasks. The scenario tests context degradation awareness and the difference between plan mode and direct execution. *The trap is believing that a larger context window solves attention distribution problems. It does not.* The “lost in the middle” effect applies regardless of total context size.

Scenario 3: Multi-Agent Research System. You are designing a multi-agent system where a coordinator dispatches research tasks to specialized subagents with iterative refinement loops. *The trap is the “super agent” anti-pattern: one agent with all 18 tools instead of distributing 4–5 tools per specialized subagent.* This scenario also tests whether you understand that subagents do not inherit coordinator context automatically. You must pass context explicitly.

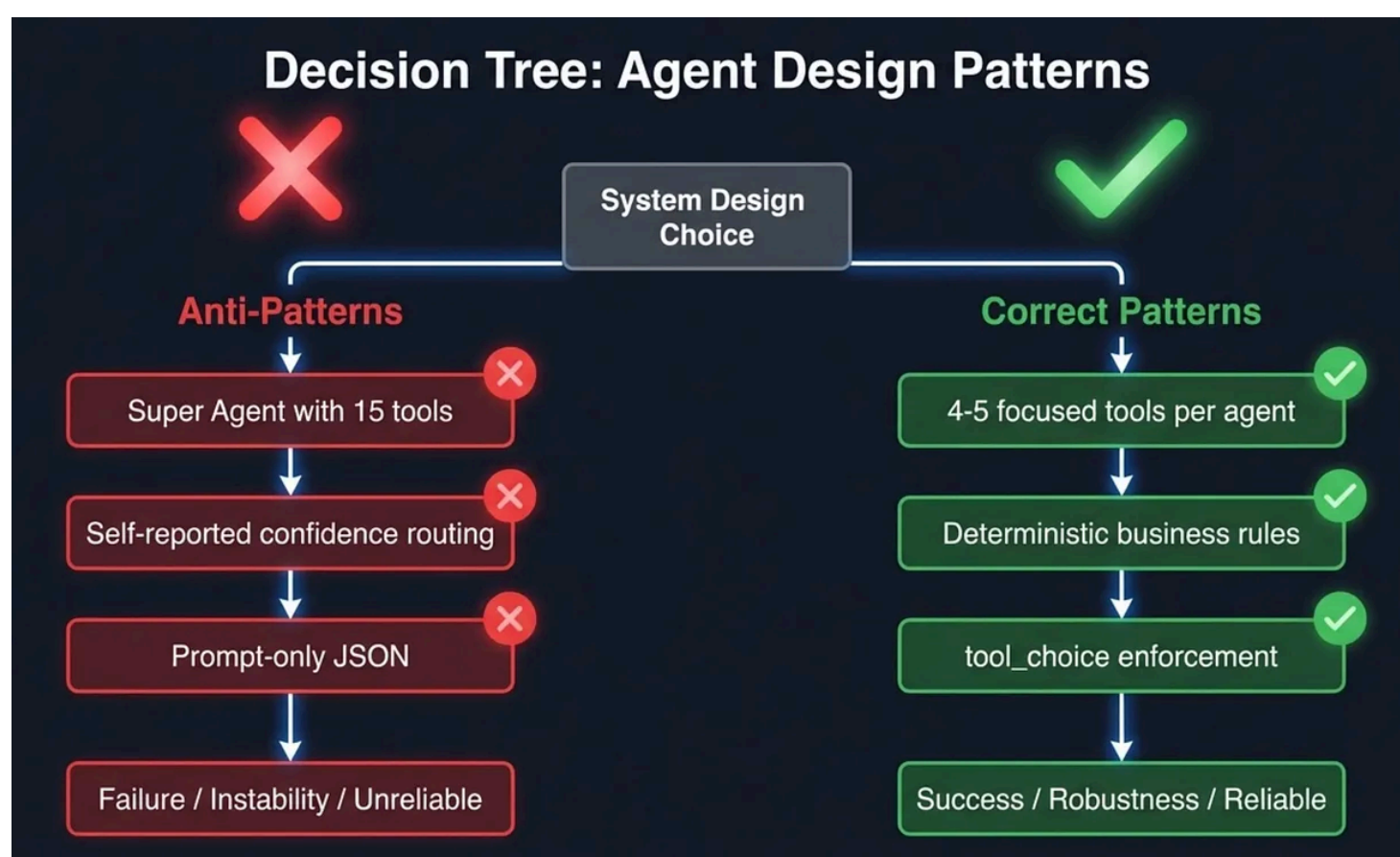
Scenario 4: Developer Productivity Tools. You are integrating Claude Code into development workflows using custom slash commands and [CLAUDE.md](#) configurations. *This scenario tests your knowledge of plan mode versus direct execution, the [CLAUDE.md](#) hierarchy (project-level vs. user-level), and how to design reusable skills.*

Scenario 5: Claude Code for CI/CD. You are integrating Claude into CI/CD pipelines for automated code reviews, test generation, and pull request feedback. *The critical tested pattern is the `-p` flag for non-interactive mode with `--bare` for reproducibility. The `--output-format json` flag for structured pipeline output is also tested.* The trap is any solution that assumes interactive mode works in a CI/CD pipeline.

Scenario 6: Structured Data Extraction. You are extracting structured data from unstructured documents using JSON schemas, handling edge cases, and integrating with downstream systems. *The trap is prompt-only enforcement. The correct approach uses `tool_choice` with input schemas or the structured outputs API with programmatic validation and retry loops.*

Critical Mental Models for the Exam

These are the principles that separate candidates who pass from those who do not. Internalize all five. They are not independent ideas; they are a coherent framework that applies everywhere on the exam.



Anti-pattern vs correct pattern decision tree showing red X paths versus green checkmark paths for five key architectural decisions

Mental Model 1: Programmatic Enforcement Beats Prompt-Based Guidance

Every time. For business rules, JSON compliance, routing decisions: do not ask Claude to follow rules in a prompt. Enforce them in code.

The mental model to carry into the exam: prompts are guidance. Code is law. When the question asks how to ensure something happens reliably, the answer involves code, not better prompt wording.

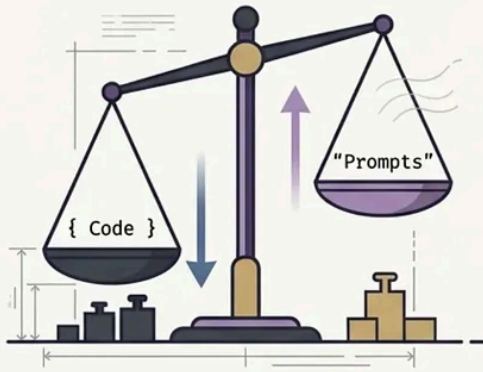
The Anti-Pattern Matrix: How to Spot Trap Answers	
Trap Pattern	Architectural Fix
15+ Tools on one Agent	4-5 tools per specialized subagent.
Using Claude Code in CI/CD interactively	Enforce the <code>`-p`</code> flag for headless execution.
'Respond only in JSON' written in the prompt	<code>`tool_choice`</code> + structured outputs <u>API</u> .
Exposing read-only data as a Tool	Expose read-only data as an <u>MCP Resource</u> .
Using the <u>Batch API</u> to save money on live support	Use the <u>Real-Time API</u> with Prompt Caching.

Spot Anti-patterns

Mental Model 2: Subagents Do Not Inherit Context

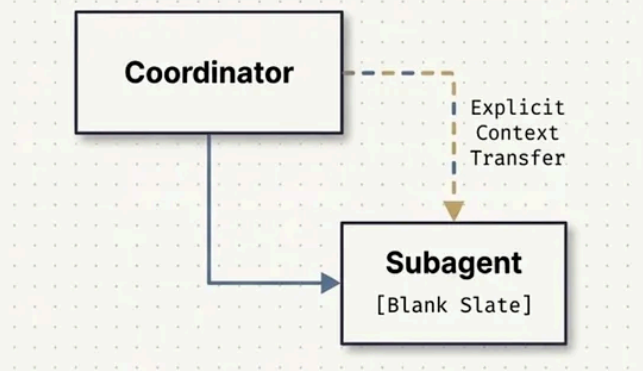
When a coordinator spawns a subagent, the subagent starts with a blank slate. You must explicitly pass the context it needs. This is one of the most heavily tested concepts on the entire exam because it is counterintuitive. Developers assume that agents operating in the same system share awareness. They do not.

The Architect's Brain: Models 1 & 2



Model 1: Programmatic Enforcement > Prompts.

For business rules, routing, and JSON compliance, code is law. Prompts are just guidance. If an answer relies on 'better prompt wording' for a strict rule, it is wrong.



Model 2: Subagents Do Not Inherit Context.

A heavily tested trap. Subagents start completely blank. When a coordinator spawns a subagent, you must explicitly pass everything it needs to know.

Mental Models 1 and 2

Mental Model 3: Tool Descriptions Drive Routing

Claude does not pick tools based on agent names or tool names. It reads tool descriptions. A poorly written description means the wrong tool gets called. Write descriptions as if you are writing documentation for a developer who has never seen your codebase. Specific, precise, unambiguous.

Mental Model 4: The "Lost in the Middle" Effect Is Real

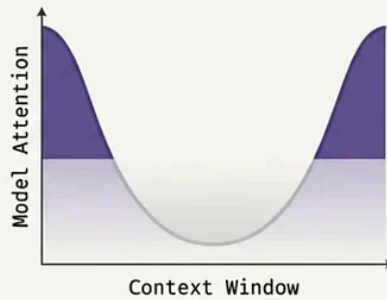
Place critical information at the beginning or end of your context. Content in the middle of a long context window receives less attention. This affects every scenario involving long context: customer support sessions, large codebase navigation, multi-turn research workflows.

The Architect's Brain: Models 3, 4, & 5



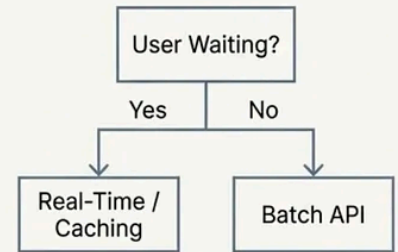
Model 3: Descriptions Drive Routing.

Claude selects tools based on precise descriptions, not agent or tool names. Write descriptions like documentation for a new developer.



Model 4: "Lost in the Middle" is Real.

Transformer-based LLMs drop attention in the middle of long contexts. Place critical data at the absolute beginning or end.



Model 5: Match API to Latency.

Match the architecture to the user's waiting status. Real-time for active workflows, Batch for overnight processing.

Mental Models 3, 4, and 5

Mental Model 5: Match API to Latency Requirements

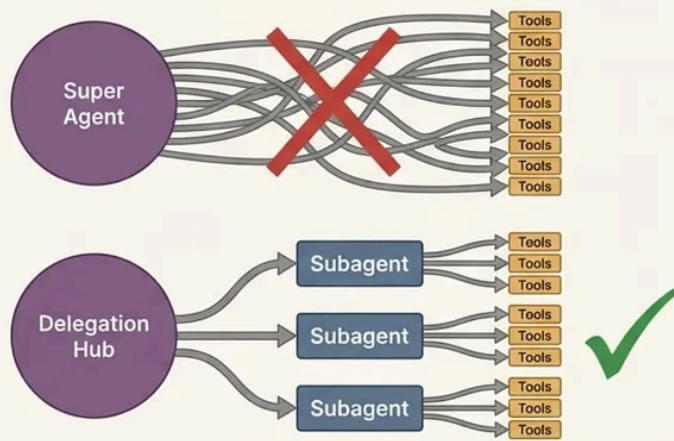
The Batch API is for background jobs. The Real-Time API is for user-facing workflows. Prompt Caching is your cost optimization strategy for repeated context with real-time requirements. When you see a cost optimization question, ask first: is anyone waiting for this response? If yes, the answer is Prompt Caching, not the Batch API.

The Anti-Pattern Catalog

The exam tests what NOT to do as much as what to do. Here are the anti-patterns organized by domain, with the exam signal for each.

Technical Deep Dive: Agents & Claude Code

Domain 1: Agentic Architecture



Avoid the “Super Agent” Trap.

Do not build an agent with 15+ tools. Selection reliability degrades. Distribute to **specialized subagents** with a strict maximum of **4-5** focused tools each.

Domain 2: Claude Code Configuration

claude_config.md

Hierarchy Rules:

```
.claude/CLAUDE.md // Project/Version Control
~/.claude/CLAUDE.md // Personal/User Preferences
```

CI/CD Execution:

```
claude -p --bare --output-format json
^         ^         ^
|         |         |
|         |         |- Pipeline parsing
|         |         |- Reproducibility
|- Headless mode (Required)
```

Interactive mode will hang pipelines. Always use the **-p** flag. Add **--bare** for reproducibility.

Antipatterns for Domain 1 and Domain 2.

Domain 1: Agentic Architecture

Anti-Pattern	Why Wrong	Correct Pattern
Super agent with 15+ tools	Selection accuracy degrades; scope creep risk	4-5 focused tools per agent; subagents for the rest
Self-reported confidence for routing	LLM confidence is not calibrated; overconfident in high-confidence range	Deterministic business rules (dollar amount, tier, issue type)
Assuming subagents inherit context	Subagents start with blank context	Explicitly pass all needed context to each subagent

Agentic Architecture

Super agent with 15+ tools: When you build a single agent with 15 or more tools, selection accuracy degrades significantly and scope creep becomes a major risk. The correct pattern is to use 4–5 focused tools per agent and delegate additional functionality to specialized subagents.

Self-reported confidence for routing: Using Claude’s self-reported confidence scores to make routing decisions is problematic because LLM confidence is not calibrated for

production use and tends to be overconfident in the high-confidence range. Instead, implement deterministic business rules based on concrete factors like dollar amount, account tier, and issue type.

Assuming subagents inherit context: A common misconception is that subagents automatically inherit the context from their coordinator agent. In reality, subagents start with a blank context slate. You must explicitly pass all needed context to each subagent to ensure they have the information required to perform their tasks.

Domain 2: Claude Code

Anti-Pattern	Why Wrong	Correct Pattern
Running Claude Code without <code>-p</code> in CI/CD	Interactive mode hangs in a headless environment	Use <code>-p</code> flag; add <code>--bare</code> for reproducibility
Putting user preferences in project <code>CLAUDE.md</code>	User-level config gets pushed to all team members	User preferences go in <code>~/.claude/CLAUDE.md</code> ; project config in <code>.claude/CLAUDE.md</code>
Direct execution for complex multi-file changes	Risk of unintended changes without a plan	Plan mode for complex, multi-step operations

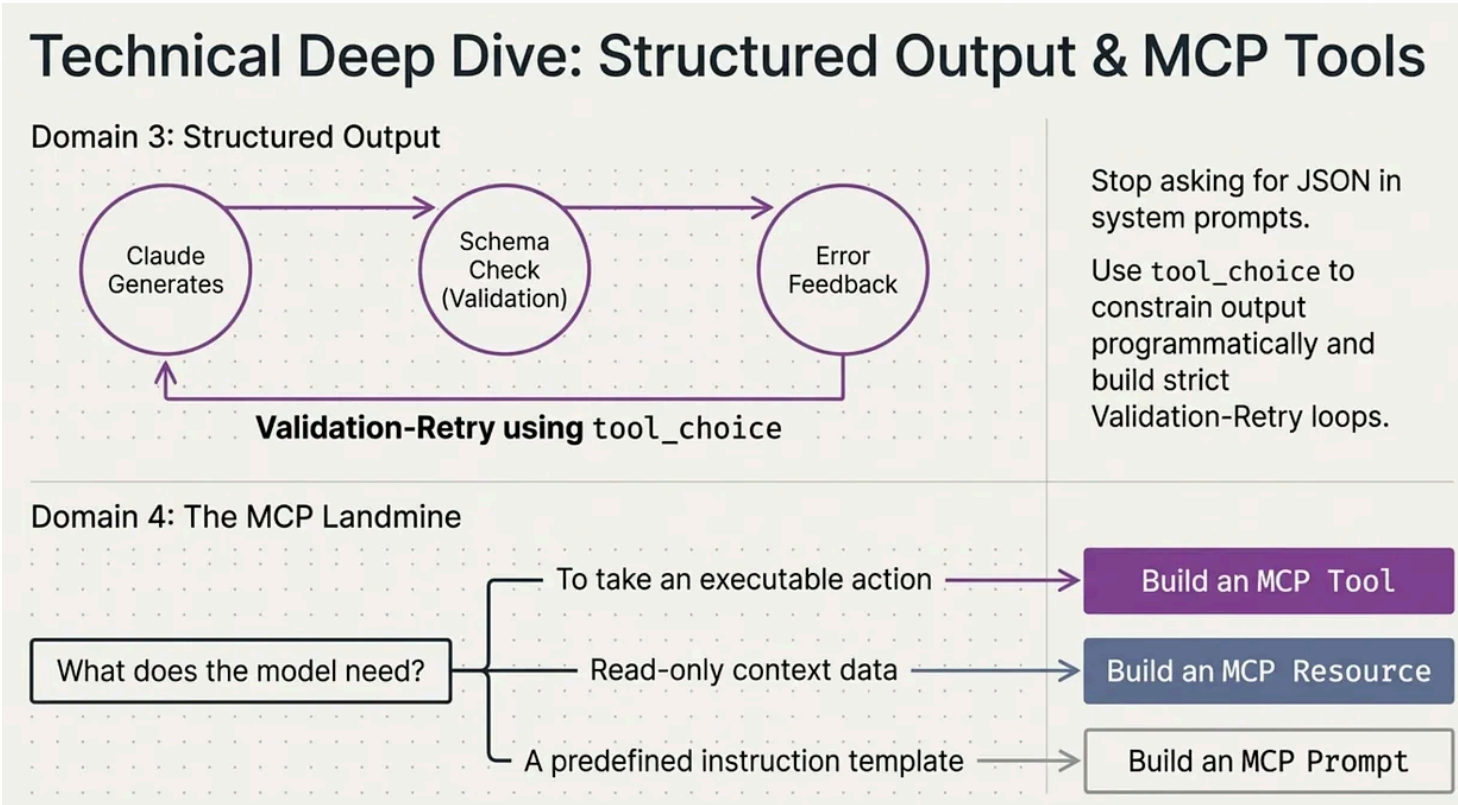
Claude Code

Running Claude Code without `-p` in CI/CD: When you run Claude Code in a CI/CD pipeline without the `-p` flag, interactive mode will hang in a headless environment because there’s no user to interact with prompts. The correct pattern is to use the `-p` flag for non-interactive mode and add `--bare` for reproducibility to ensure consistent behavior in automated environments.

Putting user preferences in project [CLAUDE.md](#): When you place user-level configuration in the project’s `.claude/CLAUDE.md` file, those personal preferences get pushed to all team members through version control, which can override their individual settings and create inconsistent development experiences. The correct pattern is to put user preferences in `~/.claude/CLAUDE.md` (the user-level config file) and reserve the project’s `.claude/CLAUDE.md` for team-wide standards and conventions that should apply to everyone working on the codebase.

Direct execution for complex multi-file changes: Using direct execution mode for complex operations that span multiple files creates a risk of unintended changes without a clear plan or opportunity for review. The model might make changes across your codebase without showing you what it intends to do first. The correct pattern is to use plan mode for complex, multi-step operations. This allows Claude Code to present a structured plan of all the changes it will make, giving you the opportunity to review and approve before any files are modified.

Domain 3: Prompt Engineering



tool_choice and validation retry loop

Anti-Pattern	Why Wrong	Correct Pattern
Prompt-only JSON enforcement	Best-effort; fails on edge cases	tool_choice with input schemas; structured outputs API
No validation-retry loop	First attempt may fail schema validation; no recovery	Validate output; retry with error context on failure

Prompt-only JSON enforcement: Relying solely on prompts to enforce JSON output is a best-effort approach that fails on edge cases. While you can instruct Claude in a prompt to “always return valid JSON,” this does not guarantee compliance, especially when handling unusual inputs or complex schemas. The correct pattern is to use `tool_choice` with input schemas or the structured outputs API, which programmatically enforces the output format and ensures reliable JSON generation.

No validation-retry loop: When you make a single API call without any validation or retry mechanism, the first attempt may fail schema validation, and you have no way to recover from the error. This leaves your system brittle and unable to handle cases where the model’s output doesn’t quite match your requirements. The correct pattern is to validate the output programmatically after each API call and, if validation fails, retry the request with error context that explains what went wrong. This creates a robust feedback loop that significantly improves reliability.

Domain 4: Tool Design

Anti-Pattern	Why Wrong	Correct Pattern
Exposing read-only data as a tool	Tools are for actions; read-only data is a resource	Use MCP Resources for data; Tools for executable actions
Vague tool descriptions	Claude cannot reliably select the right tool	Precise, specific descriptions that state exactly what the tool does and when to use it

Tool Design

Exposing read-only data as a tool: When you expose read-only data as a tool in your MCP server, you are misusing the tool abstraction. Tools are designed for executable actions that change state or perform operations, not for passive data retrieval. This creates confusion in the model’s selection logic and wastes tool call overhead on what should be simple data access. The correct pattern is to use MCP Resources for read-only data and reserve Tools for executable actions. Resources are specifically designed for static or semi-static data that the model needs to read, while tools are for operations that do something.

Vague tool descriptions: When you write vague or generic tool descriptions like “handles customer requests” or “processes data,” Claude cannot reliably select the

right tool because it lacks the specific information needed to differentiate between similar tools. The model reads tool descriptions to decide which tool to invoke, and ambiguous descriptions lead to incorrect tool selection and routing failures. The correct pattern is to write precise, specific descriptions that state exactly what the tool does, what inputs it expects, and when to use it. Treat tool descriptions as if you are writing documentation for a developer who has never seen your codebase — be explicit about boundaries, constraints, and use cases.

Domain 5: Context Management

Anti-Pattern	Why Wrong	Correct Pattern
Batch API for live customer support	Up to 24-hour processing; customers cannot wait	Real-Time API for all user-facing workflows
Raw conversation transcript as escalation handoff	Lost-in-the-middle risk; human agents need structured data	Structured JSON summary with key fields at the top
Critical info buried in middle of context	Gets less attention from the model	Place critical info at beginning or end

Context Management

Batch API for live customer support: Using the Batch API for live customer support scenarios is problematic because the Batch API can take up to 24 hours to process requests, and customers in a support interaction cannot wait that long for a response. The Batch API is designed for background jobs and non-urgent workloads where latency is not a concern. The correct pattern is to use the Real-Time API for all user-facing workflows where someone is actively waiting for a response, ensuring immediate interaction and acceptable response times.

Raw conversation transcript as escalation handoff: When you escalate from an AI agent to a human agent by simply passing the raw conversation transcript, you create two problems: first, the “lost in the middle” effect means that critical information buried in a long transcript may be overlooked by the human agent; second, human agents need structured, actionable data to resolve issues quickly, not a wall of unstructured text to parse. The correct pattern is to provide a structured JSON summary with key fields at the top — customer information, issue category, attempted

solutions, current state, and any blocking factors — so the human agent can immediately understand the situation and take action.

Critical info buried in middle of context: When you place critical information in the middle of a long context window, it receives less attention from the model due to the “lost in the middle” effect, a well-documented phenomenon where language models pay more attention to information at the beginning and end of their context. This can cause the model to miss important details or make decisions without considering key constraints. The correct pattern is to place critical information at the beginning or end of your context, ensuring it receives the attention it deserves and is properly weighted in the model’s reasoning process.

Technical Deep Dive: The Token Economics Matrix

API Feature	Cost Profile	Latency Profile	Primary Use Case
Prompt Caching	Up to 90% savings	Real-time	Repeated system prompts, policy docs, few-shot examples.
Message Batches API	50% savings	Up to 24 hours	Offline bulk processing, nightly audits, non-blocking jobs.
Real-Time API	Standard pricing	Real-time	Active user-facing, blocking workflows.

The Escalation Rule:

Deterministic rules (dollar amounts, ticket categories) trigger human handoffs. Model self-confidence scores do not.

Token Economics

Your 4-Week Study Plan

This plan assumes you are already building with Claude. If you are starting from scratch, add 2–4 weeks.

The plan is organized around where candidates lose points, not just around domain weights. Domain 1 and Domain 4 are where most people are surprised by how hard the questions actually are. Build for that.

The Execution Blueprint: Weeks 1 & 2

Week 1

Week 1: Foundations & Mental Models

Master the vocabulary.

Goal: Define agentic loops, context forking, and stop reasons without looking them up.

Key Resource: [Claude 101](#).

Week 2

Week 2: Core Skills

Highest Investment. Shift from reading to building.

Goal: Build a validation-retry loop from scratch and practice `tool_choice` with JSON schemas.

Key Resource: [Building with the Claude API](#).

Tactical Note: Reading course material without writing code leaves you with zero intuition for scenario-based questions.

Week 1 and Week 2 for the study plan

Week 1: Foundations and Mental Models

Courses: [Claude 101](#) and [AI Fluency: Framework and Foundations](#)

Goal: internalize the vocabulary and mental models before touching the advanced material.

Learn the AI Fluency Framework (Delegation, Description, Discernment, Diligence). Get comfortable with the core vocabulary: agentic loops, context windows, stop reasons, `tool_choice`. Do not rush past this. Candidates who struggle with Week 3 material often have vocabulary gaps from Week 1.

By the end of Week 1, you should be able to define without looking anything up: agentic loop, coordinator-subagent pattern, context forking, stop reason, `tool_choice`, validation-retry loop, and the “lost in the middle” effect.

Week 2: Core Skills (Highest Investment Week)

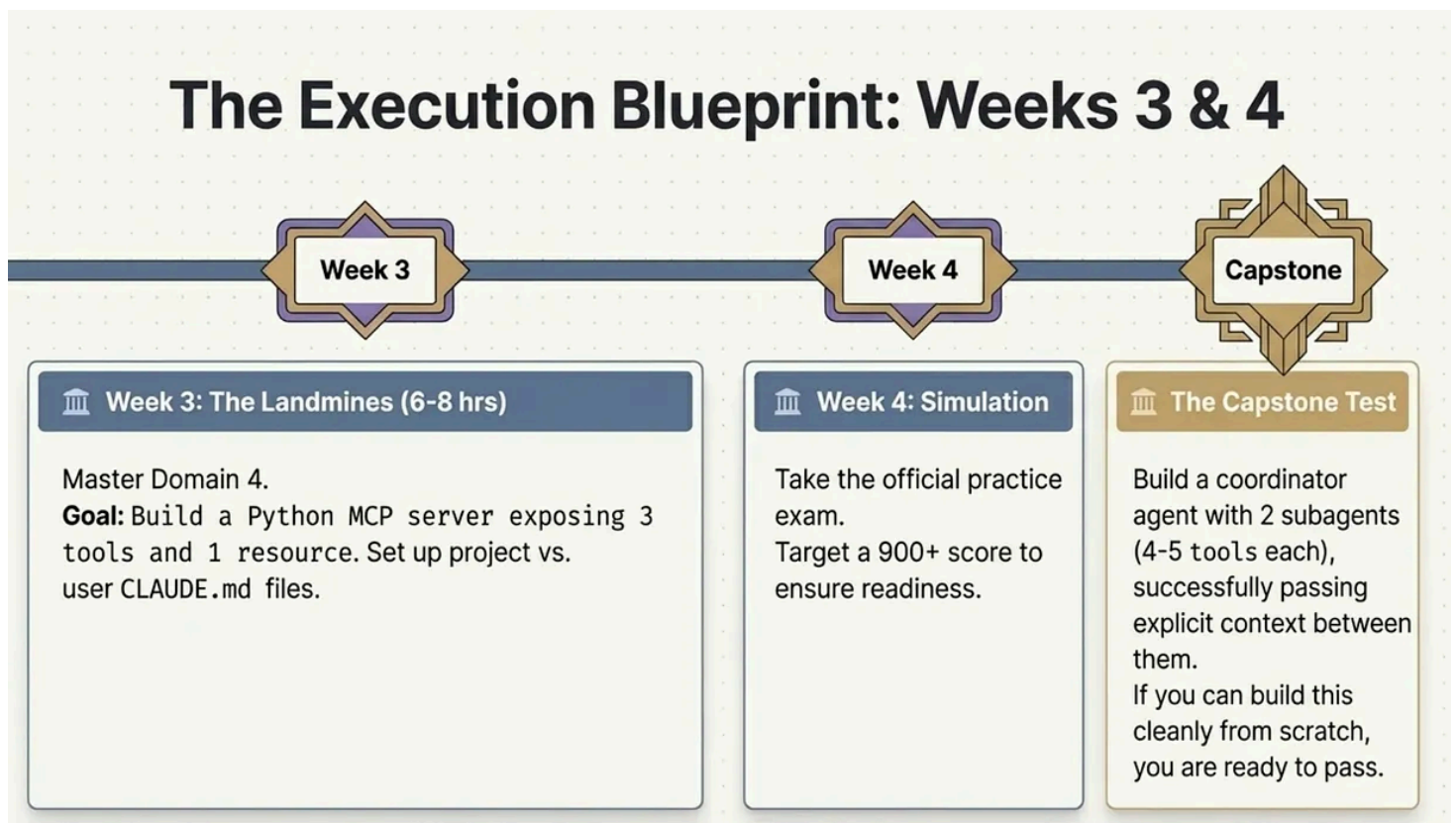
Course: [Building with the Claude API](#) (8–10 hours; this is the single most important course)

Goal: hands-on experience with the patterns the exam tests most heavily.

Work through every exercise. Build a validation-retry loop from scratch. Practice structured output with `tool_choice` and JSON schemas. Understand the `tool_use` stop reason and how to inspect tool results programmatically.

Do not skip the exercises. Reading the course material without building anything will leave you with conceptual understanding but no intuition. The exam questions are scenario-based, and intuition built from building is what makes the right answer feel obvious.

By the end of Week 2, you should have working code for: a validation-retry loop, a `tool_choice`-enforced structured output call, and a simple agent with 3-4 tools.



Week 3 and Week 4

Week 3: Advanced Topics (The Domain 4 Landmine Week)

Courses: [MCP Mastery](#), [Claude Code in Action](#), [Introduction to Agent Skills](#)

Goal: close the gaps in MCP tool design and Claude Code configuration that trip up even experienced candidates.

Build an MCP server in Python. Expose 3 tools and 1 resource. Connect it to Claude Code. Pay explicit attention to the boundary between tools and resources: every time you decide whether something should be a tool or a resource, write down your reasoning. That discipline is what makes tool boundary questions answerable on exam day.

Set up a [CLAUDE.md](#) file in a real project. Configure it with project standards. Then create a user-level config at `~/ .claude/CLAUDE.md` and make sure you understand which settings belong in each file. Run Claude Code with the `-p` flag in a CI/CD pipeline simulation. Create a custom slash command.

By the end of Week 3, you should have: a working MCP server with both tools and a resource, a real project with a properly configured [CLAUDE.md](#), and a successful Claude Code run in headless mode.

Week 4: Practice, Anti-Patterns, and Exam Simulation

Activities: Take the [official practice exam](#). Review every wrong answer. Aim for 900+ before scheduling the real exam.

Goal: convert knowledge into exam-day execution speed.

Take the practice exam under real conditions: no notes, no docs, no browser tabs. Time yourself. When you finish, categorize your wrong answers by domain. If you got more than 2 wrong in any single domain, go back to that week's material and rebuild the concept from the exercise level up.

Build the capstone project: a coordinator agent with two subagents, each with 4–5 tools, passing explicit context between them. If you can build this cleanly from scratch without referencing the documentation, you understand Domain 1 well enough to pass.

Review the anti-pattern catalog above one more time. On exam day, the fastest path to a correct answer is recognizing the trap in the wrong answers before you even read the right answer carefully.

Course Priority Matrix

Course	Maps to Domain	Priority	Study Hours
Building with the Claude API	Prompt Engineering, Tool Design	Must-do	8-10
MCP Mastery	Tool Design and MCP Integration	Must-do	4-6
Claude Code in Action	Claude Code Configuration	Must-do	3-4
Claude 101	All (Foundations)	Recommended	2-3
AI Fluency for Professionals	Mental models, Context	Recommended	2-3
Introduction to Agent Skills	Claude Code Workflows	Nice-to-have	1-2

Study Hours by Domain Weight

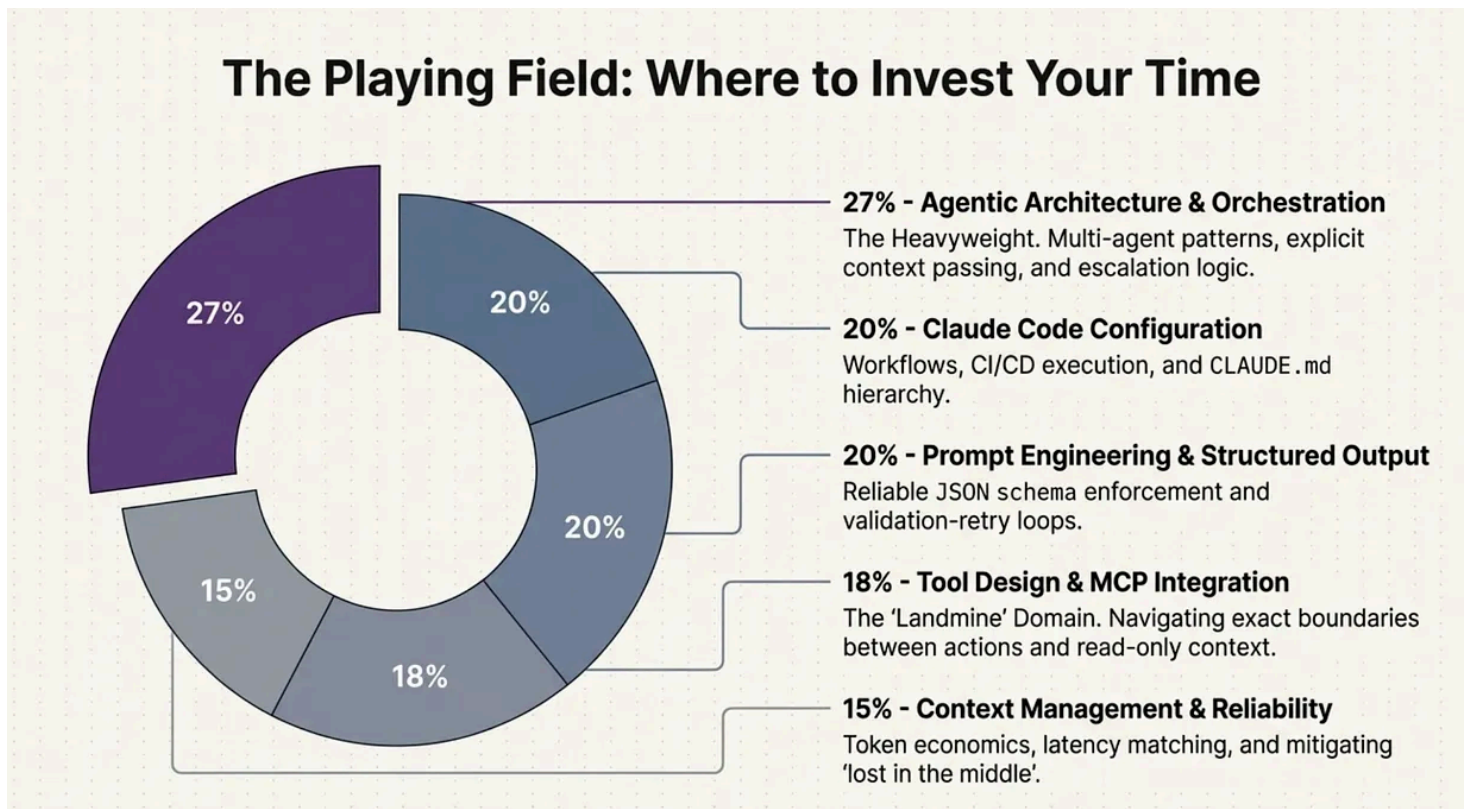
Allocate study time proportionally to domain weight, but add extra time to Domain 4 (Tool Design). The exam weight is 18%, but candidate feedback consistently cites it as the domain where they lose the most unexpected points.

Domain	Weight	Recommended Hours	Extra Note
Agentic Architecture and Orchestration	27%	8-10 hours	Highest priority; build the coordinator-subagent project
Claude Code Configuration and Workflows	20%	6-7 hours	CI/CD and CLAUDE.md hierarchy are heavily tested
Prompt Engineering and Structured Output	20%	6-7 hours	Build the validation-retry loop; do not skip it
Tool Design and MCP Integration	18%	6-8 hours	Add extra time here; tool vs. resource boundary is harder than the weight suggests
Context Management and Reliability	15%	4-5 hours	Token economics table must be memorized
Total	100%	30-37 hours	

Study Time Allocation by Domain

- Agentic Architecture and Orchestration (27%) — 8–10 hours — Highest priority; build the coordinator-subagent project

- **Claude Code Configuration and Workflows (20%)** — 6–7 hours — CI/CD and [CLAUDE.md](#) hierarchy are heavily tested
- **Prompt Engineering and Structured Output (20%)** — 6–7 hours — Build the validation-retry loop; do not skip it
- **Tool Design and MCP Integration (18%)** — 6–8 hours — Add extra time here; tool vs. resource boundary is harder than the weight suggests
- **Context Management and Reliability (15%)** — 4–5 hours — Token economics table must be memorized
- **Total: 100%** — 30–37 hours



Where to invest your time?

Early Candidate Feedback

The exam launched 11 days ago. Here is what early test-takers are reporting.

Frontline Intelligence: Early Test-Taker Feedback

Real-world insights and strategic warnings from initial exam cohorts.

The Difficulty is Real

“The distractors are highly plausible. You cannot simply watch a tutorial and pass; deep architectural intuition is required.”

Anti-Patterns Dominate

“The exam tests what not to do just as heavily as the happy path. Trap recognition is the fastest way to the correct answer.”

Domain 4 is the Gambler's Ruin

“Even experienced developers trip over MCP boundaries. If you haven't built an MCP server, you are gambling.”

Time Management

“Scenarios feature multi-part question chains where context carries over. Do not get bogged down; flag paralyzing questions and move forward.”

Frontline Test Takers

The difficulty is real. Community consensus is that this is “not a watch-a-tutorial-and-pass certification.” The depth on agentic architecture, MCP tool integration, and multi-agent orchestration is substantial. One common refrain: “The distractors are very plausible.”

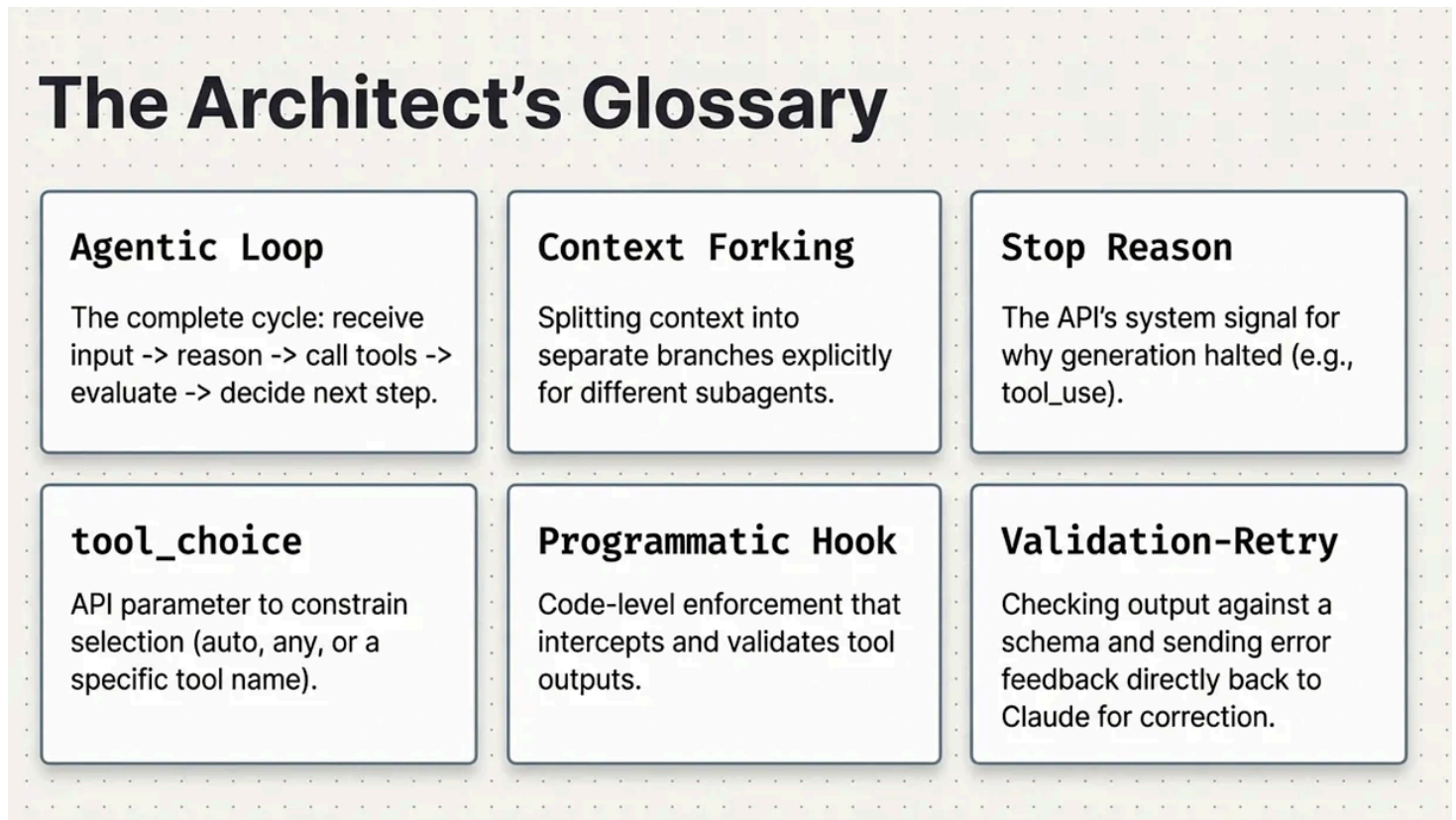
Anti-pattern recognition matters as much as knowing the right answers. Multiple candidates report that the exam tests what NOT to do as heavily as what to do. If you only study the happy path, you will miss questions about common mistakes. The anti-pattern catalog in this article is not optional study material.

MCP tool boundaries trip people up more than any other topic. Domain 4 (Tool Design and MCP Integration) is consistently cited as where people lose the most unexpected points. Knowing when to use a tool versus a resource versus a prompt requires judgment you can only build through practice. If you have not built an MCP server by the time you take the exam, you are gambling on this domain.

Time management is critical, especially on scenario chains. Some scenarios have multi-part question chains where context from one question carries into the next. Read those chains carefully. Do not get stuck. Flag and move on.

One candidate posted 985/1000 on Reddit. So perfect scores are possible. The passing score is 720, which gives you meaningful room for error. You do not need to know

everything perfectly. You need to know the critical patterns reliably and recognize the traps quickly.



Architect's Glossary

Key Terminology Glossary

These terms are testable. Know them precisely, not just approximately.

- **Agentic Loop:** The cycle: receive input, reason, call tools, evaluate results, decide next step
- **Context Forking:** Splitting context into separate branches for different subagents
- **Coordinator-Subagent Pattern:** A coordinator agent delegates to specialized subagents and synthesizes results
- **Programmatic Hook:** Code-level enforcement (e.g., `PostToolUse`) that validates or transforms tool outputs
- **Stop Reason:** The API's signal for why generation stopped (e.g., `tool_use`, `end_turn`, `max_tokens`)
- **tool_choice:** API parameter controlling tool selection: `auto` (model decides), `any` (must use a tool), or specific tool

- **Validation-Retry Loop:** Check output against schema; on failure, send error back to Claude for correction
- **MCP Resources:** Data catalogs/schemas exposed for ingestion (read-only context)
- **MCP Tools:** Executable functions the model can invoke to take actions
- **MCP Prompts:** Predefined instruction templates or workflows

Preparation Resources

The Arsenal: Required Resources

Official (Anthropic Academy)

Claude 101

API Building

MCP Mastery

Practice Exam

Required Documentation

Agent SDK Fira Code

Claude Code Fira Code

MCP Fira Code

Batch Processing

Structured Outputs

Community Intelligence

DEV Community Roadmaps

FlashGenius Flashcards

Udemy Practice Tests

Final Directive: The knowledge has to live in your head, not in a browser tab. Go build.

Preparation Resources

Recommended Resources to Study for the Claude Certified Architect Foundation Exam

To get the most out of this practice exam, study the full CCA Exam Prep series in order:

1. [Claude Certified Architect: The Complete Guide to Passing the CCA Foundations Exam](#) The series opener and exam roadmap. It covers the CCA Foundations format, domain weights, scenario types, and a practical study plan for passing on the first try.

2. [Claude Certified Architect — Exam Prep: Mastering the Customer Support Resolution Agent Scenario](#) A deep dive into the customer support scenario, focused on escalation rules, compliance workflows, and why deterministic business logic beats model self-confidence in production support systems.
3. [CCA Exam Prep: Mastering the Code Generation with Claude Code Scenario](#) Focuses on code-generation questions from the exam, especially context degradation, CLAUDE.md hierarchy, large-codebase refactoring, and CI/CD-safe Claude Code usage.
4. [CCA Exam Prep: Structured Data Extraction](#) Covers high-reliability extraction pipelines, including JSON schema enforcement, semantic validation, retry loops, and the exam's preference for programmatic guarantees over prompt-only approaches.
5. [CCA Exam Prep: Mastering the Multi-Agent Research System Scenario](#) Explains the multi-agent research scenario through hub-and-spoke orchestration, context isolation, tool scoping, explicit context passing, and failure handling patterns that the CCA exam heavily tests.
6. [Claude Certified Architect: Master the CI/CD scenario for the CCA Foundations Exam](#) This article focuses on the CI/CD scenario in the CCA series, covering the flags and pipeline patterns that matter in production. It explains why -p, — bare, and structured JSON output are central to passing exam questions about non-interactive automation, validation loops, and reliable pipeline behavior.
7. [CCA: Master the Developer Productivity scenario for the Claude Certified Architect exam](#) This piece covers the Developer Productivity scenario, with emphasis on CLAUDE.md hierarchy, MCP configuration, tool scoping, and team workflow design. It frames developer productivity as a multi-domain CCA topic and highlights the exam's preference for programmatic enforcement, clear configuration boundaries, and disciplined agent design.
8. [Claude Certified Architect Practice Exam: 60 Questions with Detailed Explanations](#) This full-length 60-question practice exam (the article you are currently reading) tests all five domains and six scenarios with detailed explanations that teach why every wrong answer is wrong. Use it as your final readiness check.

These eight articles form a complete preparation package. Start with the Complete Guide, work through the scenario deep-dives, and finish with this practice exam to

test your readiness and identify any remaining gaps before scheduling the real CCA Foundations exam.

Exam Prep Guides From Rick Hightower

(No Paywall, friend links for my subscribers to Substack)

CCA-F Exam Prep

1. Claude Certified Architect: The Complete Guide to Passing the CCA Foundations Exam <https://pub.towardsai.net/claude-certified-architect-the-complete-guide-to-passing-the-cca-foundations-exam-9665ce7342a8?sk=a537f4d8379a5268c3abf806c869ef32>
2. Claude Certified Architect — Exam Prep: Mastering the Customer Support Resolution Agent Scenario: <https://medium.com/towards-artificial-intelligence/claude-code-certification-exam-prep-mastering-the-customer-support-resolution-agent-scenario-5b82a086eaf8?sk=8858eae08560609ef23ae9eea6b36ea>
3. CCA Exam Prep: Mastering the Code Generation with Claude Code Scenario: <https://medium.com/towards-artificial-intelligence/cca-exam-prep-mastering-the-code-generation-with-claude-code-scenario-95f2d8d06742?sk=e4fdc266386579e049ff626b485bc719>
4. CCA Exam Prep: Structured Data Extraction: <https://medium.com/towards-artificial-intelligence/cca-exam-prep-structured-data-extraction-86ad3c7541a3?sk=184864e65689f61be5c3c8c9259145b3>
5. CCA Exam Prep: Mastering the Multi-Agent Research System Scenario: <https://medium.com/towards-artificial-intelligence/cca-exam-prep-mastering-the-multi-agent-research-system-scenario-aa0c446a5e7d?sk=229a3cb07492ccf256c30c65eecf0f13>
6. Claude Certified Architect: Master the CI/CD scenario for the CCA Foundations Exam — the flags, output formats, and pipeline patterns that separate passing: <https://medium.com/towards-artificial-intelligence/claude-certified-architect-master-the-ci-cd-scenario-for-the-cca-foundations-exam-the-flags-de2478a346da?sk=bce6cee4bba262d0328f892a4b80fbe5>

7. CCA: Master the Developer Productivity scenario for the Claude Certified Architect exam — from CLAUDE.md hierarchies to MCP scoping to the anti-patterns that trip up most candidates: <https://medium.com/towards-artificial-intelligence/cca-master-the-developer-productivity-scenario-for-the-claude-certified-architect-exam-from-e402d9bb277d?sk=f2c73c9b0165601a9ff5e1242e25f3ff>
8. Claude Certified Architect Practice Exam: 60 Questions with Detailed Explanations: <https://medium.com/towards-artificial-intelligence/claude-certified-architect-practice-exam-60-questions-with-detailed-explanations-3a4d2267603d?sk=e439ed89b4ca8b52cab8866adde84027>
9. Production Claude Agents: 6 CCA-Ready Patterns That Make LLMs Actually Obey Business Rules (Full Code + Notebooks): <https://medium.com/towards-artificial-intelligence/production-claude-agents-6-cca-ready-patterns-that-make-llms-actually-obey-business-rules-full-c2cf28b7669f?sk=11e913aceb0fb85d6621804d41a21f55>
10. CCA-F Exam Prep: The Multi-Agent Research System in Runnable Code: <https://medium.com/towards-artificial-intelligence/cca-f-exam-prep-the-multi-agent-research-system-in-runnable-code-2dcfab09c296?sk=26bfaae78e007f797c6b54bbb4beaccf>
11. Foundations of CCA-F Exam: 5 Battle-Tested LLM Agent Patterns (No Bloat Required): <https://medium.com/towards-artificial-intelligence/foundations-of-cca-f-exam-5-battle-tested-llm-agent-patterns-no-bloat-required-4e3ad4037e3f?sk=d006885b6a9a91c9d546bcae4e121726>
12. Domain 1: Architecting Production-Grade Agents through LLM Orchestration and Agentic Loops: <https://medium.com/towards-artificial-intelligence/architecting-production-grade-agents-through-llm-orchestration-and-agentic-loops-d2f330e28224?sk=669dce953823a39a5520e11a78358142>
13. The Eleven Patterns Behind Every Production Agentic System (And Where JSON Schemas Actually Earn Their Keep): <https://medium.com/towards-artificial-intelligence/the-eleven-patterns-behind-every-production-agentic-system-and-where-json-schemas-actually-earn-48c1e8f8173c?sk=b51bbf2c2433f1ddab08bc58a446406b>
14. Claude Agent SDK: The Coordinator That Forgets to Check Its Work: Iterative Refinement Loops in Multi-Agent Systems (Domain 1.6): <https://medium.com/towards-artificial-intelligence/claude-agent-sdk-the->

[coordinator-that-forgets-to-check-its-work-iterative-refinement-loops-in-7f222fa15006?sk=ba45e6534f5212137112549db42ebdc9](https://medium.com/towards-artificial-intelligence/coordinator-that-forgets-to-check-its-work-iterative-refinement-loops-in-7f222fa15006?sk=ba45e6534f5212137112549db42ebdc9)

15. The Memory Leak in Your AI Strategy: Architecting for LLM Reliability at Scale: CCA-F Exam Prep Domain 5 — <https://medium.com/towards-artificial-intelligence/the-memory-leak-in-your-ai-strategy-architecting-for-llm-reliability-at-scale-ec01eaa02d04?sk=bc034e25d1de0b86bf745de82063458c>
16. The Reliability Hierarchy: From Probabilistic Prompts to Deterministic Engineering: CCA-F Domain 4: Why “Asking Nicely” Fails in Production, and How to Architect Guaranteed Results — <https://medium.com/towards-artificial-intelligence/the-reliability-hierarchy-from-probabilistic-prompts-to-deterministic-engineering-0b5969c7c359?sk=fbaeac33d348fa018cbcd55a98fe5bae>
17. Engineering Dynamic Context: The Claude Code Architecture That Survives Production—CCA-F Domain 3: From Token Exhaustion to Production-Grade Workflows: <https://medium.com/towards-artificial-intelligence/engineering-dynamic-context-the-claude-code-architecture-that-survives-production-77ee1f2b0e45?sk=5b21463f621cc0f10628b7f72beca3a6>
18. The Architect’s Blueprint: Why Your AI Agent Keeps Picking the Wrong Tool (And How to Fix It) — CCA-F Domain 2: Beyond prompt-patching: A senior engineer’s guide to structural reliability, tool boundaries, and the Model Context Protocol: <https://medium.com/towards-artificial-intelligence/the-architects-blueprint-why-your-ai-agent-keeps-picking-the-wrong-tool-and-how-to-fix-it-4abf695dddb7?sk=a4f8a84dff6decf1796f0ca584d461f1>

Official Anthropic Resources

- Anthropic Academy: anthropic.skilljar.com (13 courses, FREE for everyone; no partner access needed)
- CCA Exam Guide: [Official exam guide on SlideShare](#)
- Practice Exam: Available through Anthropic Academy (benchmark: score 900+ before taking the live exam)
- Exam Registration: [Access request form](#)

Official Documentation to Study

- [Claude Agent SDK Documentation](#)

- [Claude Code Documentation](#)
- [MCP Documentation](#)
- [Advanced Tool Use \(Anthropic Engineering\)](#)
- [Batch Processing Documentation](#)
- [Structured Outputs Documentation](#)

Community Resources

- [DEV Community: Inside the CCA Program](#)
- [DEV Community: CCA Preparation Roadmap](#)
- [FlashGenius: CCA Flashcards and Guide](#)
- [Claude Certifications Study Site](#)
- [Udemy: CCA Practice Tests](#)

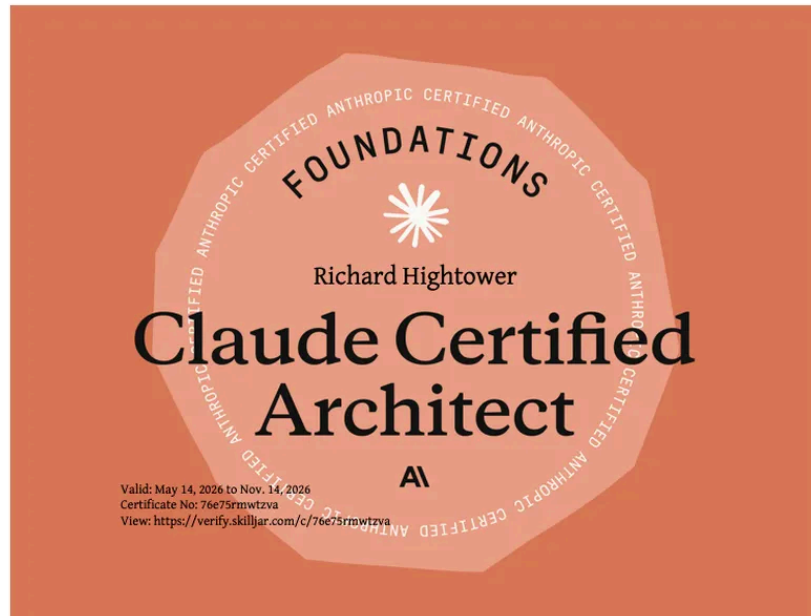
This is Part 18 of “Claude Code, Day-to-Day,” a 19-part guide to mastering Claude Code for working engineers.

About the Author — Claude Certified Architect

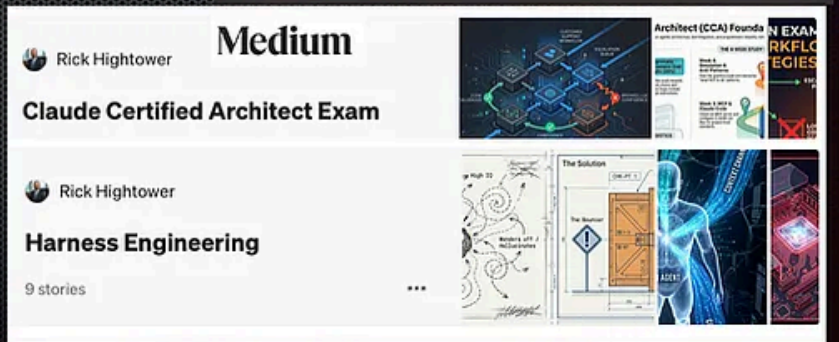
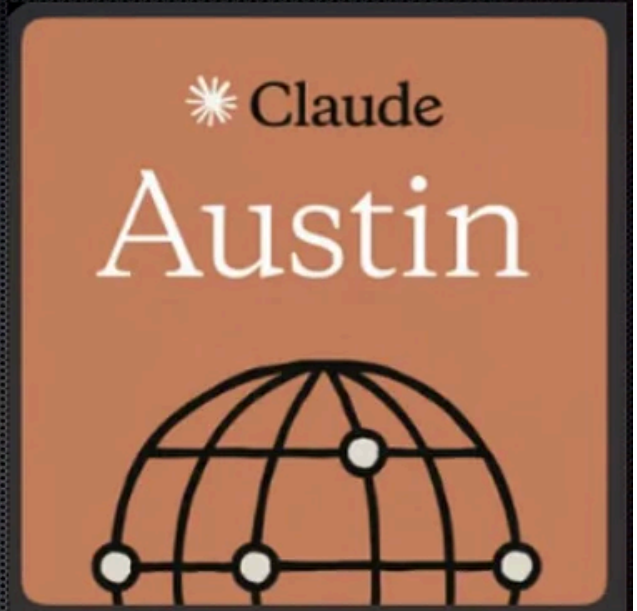
Rick Hightower is a former Senior Distinguished Engineer at a Fortune 100 company, focusing on delivering ML / AI insights to front-line applications, and a practitioner building multi-agent production systems. Rick is a Claude Certified Architect. Follow him on Medium for more hands-on agent engineering content. You can also book him to speak and train your team: Check out [Rick Hightower’s SpeakerHub](#).



Rick created Skilz, the [universal agent skill installer](#) that supports 30+ coding agents, including Claude Code, Gemini, Copilot, and Cursor, and co-founded the world's largest agentic skill marketplace. Connect with Rick Hightower on [LinkedIn](#) or [Medium](#). Check out [SpillWave](#), your source for AI expertise.



Rick Hightower is a Claude Certified Architect and working on a book for Manning on Harness Engineering.



Anthropic Harness Engineering: Author Rick Hightower, AI systems practitioner and agentic frameworks developer

Rick has been actively developing generative AI systems, agents, and agentic workflows for years. He is the author of numerous agentic frameworks and developer tools and brings deep practical expertise to teams adopting AI. He enjoys writing about himself in the 3rd person.

Rick also wrote a [Claude Certified Architect](#) (CCA) series of articles that have a lot of useful information on writing agentic AI systems. Many ideas captured in the CCA and the exam prep Rick wrote echo what you see in this article. If you want to improve your ability to create well-behaved AI agents, studying for the CCA Exam is a good place to start.

CCA Exam Prep on Agentic Development

- [Claude Certified Architect: The Complete Guide to Passing the CCA Foundations Exam](#)
- [CCA Exam Prep: Mastering the Code Generation with Claude Code Scenario](#)
- [CCA Exam Prep: Mastering the Multi-Agent Research System Scenario](#)

- [CCA Exam Prep: Structured Data Extraction](#)
- [CCA: Master the Developer Productivity Scenario](#)
- [Claude Certified Architect: Master the CI/CD Scenario](#)
- [CCA Exam Prep: Mastering the Customer Support Resolution Agent Scenario](#)
- Get the complete reading list for CCA-F exam prep articles from this [Claude Certified Architect Exam Prep list](#).

Rick also wrote a series on harness engineering and how to improve agentic systems using harness engineering for feedback loops and adversarial agents. These articles also go hand in hand with this article.

Harness Engineering Articles

- [The \\$9 Disaster: What Anthropic's Harness Design Paper Teaches Us About Building Autonomous AI](#)
- [Harness Engineering vs Context Engineering: The Model is the CPU, the Harness is the OS](#)
- [LangChain Deep Agents: Harness and Context Engineering: Memory, Skills, and Security](#)
- [Beyond the AI Coding Hangover: How Harness Engineering Prevents the Next Outage](#)
- [LangChain's Harness Engineering: From Top 30 to Top 5 on Terminal Bench 2.0](#)
- [Anthropic's Harness Engineering: Two Agents, One Feature List, Zero Context Overflow](#)
- [OpenAI's Harness Engineering Experiment: Zero Manually-Written Code](#)



Claude
Austin



Presented by

Claude Community E... >

Subscribe

Claude Community Events hosted globally by Claude Community members. Meet others building with Claude and keep thinking. Learn more:



Hosted By



Damon Bodine



Jason Desiderio

Austin | Claude Code

APR 13

Monday, April 13

6:00 PM - 8:00 PM



Capital Factory ↗

Austin, Texas

Registration



Event Full

If you'd like, you can join the waitlist.

Please click on the button below to join the waitlist. You will be notified if additional spots become available.

Join Waitlist

6:25 PM - 6:30 PM -- Welcome & Opening Remarks

Our groups will break out to dedicated sessions based on your interest.

7:05 PM - 7:45 PM |



Rick Hightower (Spillwave Solutions)

From Claude Code Skills to Adversarial Subagent Orchestrators to the Claude Agent SDK: Three Generations of a Production Research Pipeline

A special Thank you to Station Austin for sponsoring Claude Code Austin. Station Austin is the center of gravity for entrepreneurs in Texas. They bring together the best entrepreneurs in the state and connect them with their first

Hightower's AI Harness Engineering is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Type your email...

Subscribe

5 Likes • 2 Restacks

Discussion about this post

Comments Restacks

Write a comment...

